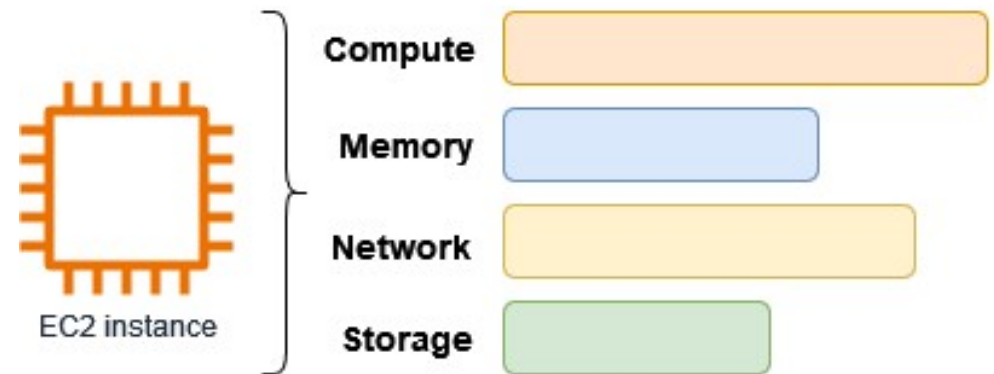


AWS EC2

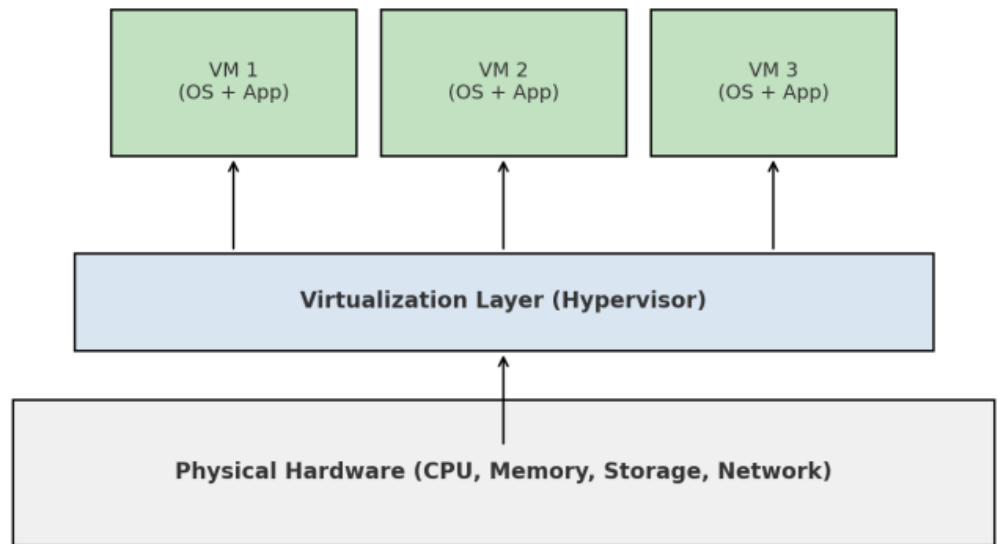
EC2 Basic

- AWS EC2 (Elastic Compute Cloud) in simple term means: **renting a virtual computer in the cloud and** is one of the most popular of AWS' offering.
- Knowing EC2 is fundamental to understand how the Cloud works.
- Its like leasing a laptop that lives in an AWS data center instead of on your desk.
- You choose the "specs" (CPU, RAM), the "operating system" (Windows, Linux), and the "hard drive" size, and you only pay for the time it's turned on.



Foundation of EC2 : Hypervisor

- Imagine you bought a **massive building of 10 floor**, but you only lived in small space. The other space empty, but you still had to pay for the heating, taxes, and cleaning for the whole house.
- The **Hypervisor** is like a master architect who goes into that building and turns it into an **Apartment Complex**.
 - **Abstraction (The Blueprint):** The hypervisor hides the "pipes and wires" of the building. You don't care how the electricity gets to your apartment; you just flip a switch and it works.
 - **Slicing (The Rooms):** It takes the big house and puts up walls. It says, "*You get these 2 rooms, and your neighbor gets those 3 rooms.*" Even though it's one big building, everyone feels like they have their own private home.
 - **Isolation (The Deadbolts):** This is the most important part. If your neighbor accidentally starts a small kitchen fire (a crash) or leaves their door unlocked (a hack), the firewalls and locks keep **your** apartment perfectly safe.
- **EC2 Instance:** Your private apartment inside that building.



AWS Hypervisor

AWS Xen (The "Software" Legacy)

- **Software-Based:** Xen is a traditional open-source hypervisor that AWS used exclusively for over a decade (the "Classic" era).
- **Management Overhead:** It requires a dedicated, "privileged" virtual machine (called **Dom0**) to handle all networking and storage tasks.
- **The "Tax":** Because Dom0 consumes some of the physical CPU and RAM for management, there is a small "virtualization tax," meaning you don't get 100% of the raw hardware performance.
- **Used For:** Older instance generations like **T2, M4, and C4**.

AWS Nitro (The "Hardware" Revolution)

- **Hardware-Accelerated:** Nitro moves the management tasks (networking, storage, security) off the main processor and onto **dedicated Nitro Cards** (physical hardware).
- **Bare-Metal Performance:** Because the main CPU no longer has to "manage" the VM, nearly **100% of the resources** are dedicated to your application.
- **The Modern Standard:** Powering all current-generation instances (e.g., **T3, M5, C5, and newer**), offering faster networking (up to 200Gbps) and lower latency.

EC2 Key Components

- **Amazon Machine Image (AMI):** is a template for your instance. It contains everything needed to boot the server.
 - **Operating System:** (e.g., Ubuntu, Amazon Linux, Windows Server).
 - **Software:** Any pre-installed applications or configurations.
- **Instance Types (The Hardware Profile):** AWS offers hundreds of instance types, categorized by their "strengths." You choose them based on what you're doing:
 - **General Purpose (T-family):** Balanced for small web servers or dev environments.
 - **Compute Optimized (C-family):** High CPU; great for batch processing or media encoding.
 - **Memory Optimized (R-family):** High RAM; best for large databases.
 - **GPU Instances (G-family):** For machine learning.
- **Security Groups(The Firewall):** is a virtual firewall that controls traffic.

EC2 Key Components

- **Elastic Block Store (EBS)** is the virtual hard drive:
 - Unlike a physical hard drive, it is "network-attached," meaning it lives separately from the CPU.
 - **Persistence:** If your instance crashes, the data on the EBS volume stays safe. You can even detach a drive from one server and plug it into another.
- **Key Pairs:** Instead of a password, AWS uses a **Key Pair** (a public key and a private key) to verify your identity.
 - **Public Key:** Stored on the EC2 instance.
 - **Private Key:** Downloaded to your computer as a .pem/ppk file. You must keep this safe; if you lose it, you are usually locked out!.
- **Auto Scaling:** The real power of EC2 isn't just that it's a virtual server; it's that it can change size automatically. You can set rules (e.g., "If CPU > 70%, add another server"). This ensures you can handle traffic spikes without paying for extra servers when it's quiet.
- **Elastic Load Balancing (ELB):** If you have 10 servers running, the ELB acts as a "traffic cop," distributing incoming visitors evenly so no single server gets overwhelmed.

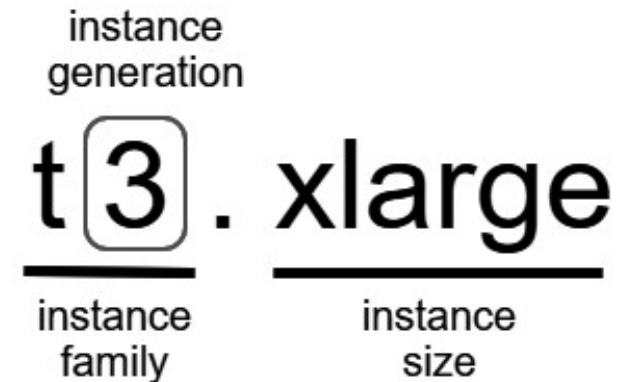
EC2 User Data

- If you wanted to launch an EC2 instance that immediately becomes a functional, User Data script can help you achieve this.
- User Data is used for
 - **bootstrapping**. It helps instance from a "blank slate" (a fresh OS) to a "ready-to-work" state without human intervention.
 - perform repetitive setup tasks—like installing updates, configuring instance for services.
 - run setup commands automatically during the boot process.
- It run **only once**, during the very first boot cycle of the instance.
- On Linux, these scripts run as the root user and uses shell script, On Windows, you use PowerShell scripts.

EC2 Instance Decoding the Name

AWS uses a specific naming convention that looks like code, but it's actually quite simple once you break it down (e.g., t3.xlarge).

- **Family (t):** The primary purpose (General Purpose).
- **Generation (3):** The version of the hardware. Higher numbers are newer and usually offer better price-performance.
- **Size (xlarge):** How much capacity (CPU/RAM) is packed into this specific instance.
- Within each family, you choose a size. These usually double in capacity as you move up the scale:
 - **nano, micro, small, medium, large, xlarge, 2xlarge ...** all the way up to **metal** (where you get the entire physical server to yourself).



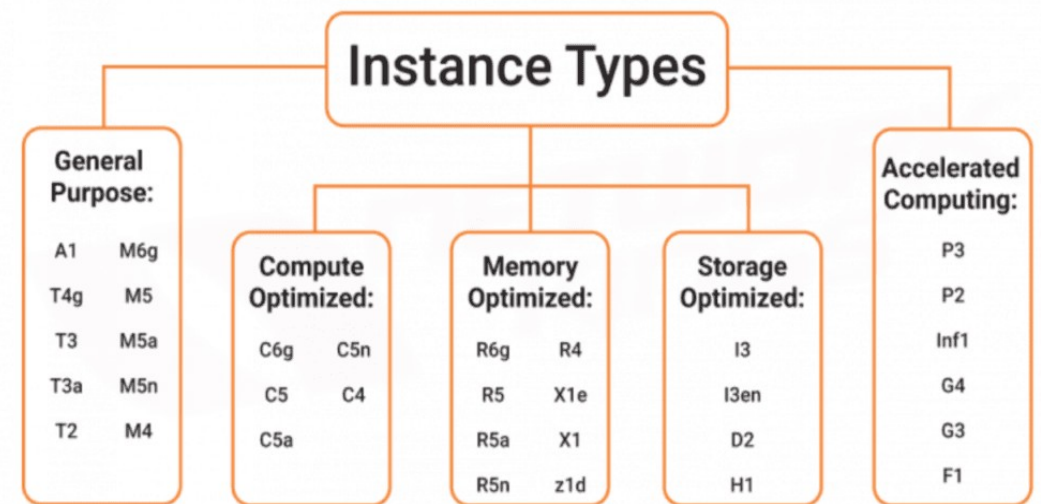
The screenshot shows the AWS Management Console 'Instance types' page. The table lists various instance types with their vCPUs, Architecture, Memory (MiB), and Storage. The 'Instance type' column is highlighted with a red box, and a red arrow points to the 't3.xlarge' instance type.

Instance type	vCPUs	Architecture	Memory (MiB)	Storage
t1.micro	1	i386, x86_64	627	-
t2.nano	1	i386, x86_64	512	-
t2.micro	1	i386, x86_64	1024	-
t2.small	1	i386, x86_64	2048	-
t2.medium	2	i386, x86_64	4096	-
t2.large	2	x86_64	8192	-
t2.xlarge	4	x86_64	16384	-
t2.2xlarge	8	x86_64	32768	-
t3.nano	2	x86_64	512	-
t3.micro	2	x86_64	1024	-
t3.small	2	x86_64	2048	-

EC2 Instance Type

AWS organizes these into five main categories.

1. General Purpose (Balance)
2. Compute Optimized (Processors)
3. Memory Optimized (RAM)
4. Accelerated Computing (Hardware Extras)
5. Storage Optimized (Data Throughput)



General Purpose - EC2 Instance Type

- **General Purpose (Balance)** These instances provide a balanced mix of compute (CPU), memory (RAM), and networking resources
 - **Families:** T4g, T3, M5, M6g
- **Best for:** Web servers, small databases, and development environments.
- The Two Sub-Types:
 - Burstable Performance (T-Series: t3, t3a, t4g)
 - **Best for:** Workloads that are mostly quiet but occasionally busy.
 - Fixed Performance (M-Series: m5, m6g, m7g)
 - **Best for:** Applications with steady, high utilization where you can't risk "running out of credits."

Compute Optimized - EC2 Instance Type

- **Compute Optimized** instances are the "High-Performance Sports Cars". They are designed for applications that need raw processing power and high-speed calculation capabilities.
 - **Families:** C5, C6g, C7g
- It prioritizes the **CPU**.
- **Best for:** High-performance web servers, batch processing, media encoding, and scientific modeling.

Memory Optimized - EC2 Instance Type

- While Compute Optimized instances are all about the engine (CPU), **Memory Optimized** instances are all about the floor space (**RAM**).
 - **Families:** R5, R6g, X1, High Memory
- **Best for:** High-performance databases (like SAP HANA), real-time big data processing, and large in-memory caches.
- Types:
 1. **R-Series (The Standard: R8g, R8i):** The "R" stands for **RAM** and are the most common memory-optimized instances.
 2. **X-Series (The Massive: X2gd, X8i, X2idn):** R-series isn't enough, you go to "X."
 3. **High Memory Instances (U7i):** For most extreme enterprise workloads (like massive SAP HANA deployments)

Accelerated Computing - EC2 Instance Type

- These instances use **hardware accelerators**—extra chips like GPUs, FPGAs, or custom AI processors—to perform specific Mathematical tasks much faster than a CPU ever could.
 - **Families:** P4, G5, F1
- **Best for:** Graphics processing, 3D rendering, machine learning (AI), and video gaming.
- Types:
 - 1. GPU Instances (NVIDIA Powered): P-Series (P5, P5en, P6) & G-Series (G6, G6e)
 - 2. AWS Silicon (Custom AI Chips): Trn-Series (Trn2, Trn3) & Inf-Series (Inf2, Inf3)
 - 3. FPGA Instances (F1, F2) Chameleon Chips

Storage Optimized - EC2 Instance Type

- These are built for massive **data throughput** and low-latency access to huge datasets stored locally on the server.
 - **Families:** I3, D2, H1
- **Best for:** NoSQL databases (like Cassandra/MongoDB), data warehousing, and log processing.
- Types:
 1. I-Series (IOPS Focus: I8g, I4i, Im4gn): The "I" stands for **I/O**. These use NVMe SSDs for extreme speed.
 2. D-Series (Density Focus: D3, D3en): The "D" stands for **Density**. These use traditional HDDs for massive capacity.
 3. H-Series (Hybrid Focus: H1): A balance between the D and I series.

EC2 Purchase Options

On-Demand Pricing (Pay-as-you-go)

- **How it works:** You pay for compute capacity by the second (minimum of 60 seconds) with no long-term commitment.
- **Best for:** New, unpredictable workloads; short-term projects; or application development/testing.
- **Pros/Cons:** Highly flexible, but it is the **most expensive** per hour.

Savings Plans (The Modern Commitment Model)

- **How it works:** You commit to a consistent amount of compute usage (measured in \$/hour, e.g., \$10/hour) for a **1 or 3-year term**.
- **Discount:** Up to **72%** off On-Demand rates.
- **Best for:** Stable, long-term production workloads.
- **Flexibility:** "Compute Savings Plans" automatically apply across any EC2 instance family, size, OS, or even AWS Region.

Reserved Instances or RIs (The Legacy Commitment Model)

- **How it works:** Similar to Savings Plans, you commit to a **1 or 3-year term**, but traditionally you commit to a specific *instance configuration* (e.g., m5.large in us-east-1).
- **Discount:** Up to **72%** off On-Demand rates.
- **Current Status:** While still available, AWS heavily pushes Savings Plans over RIs because Savings Plans are vastly easier to manage and much more flexible.

Core Difference B/w Saving & Reserved

<i>Feature</i>	<i>Reserved Instances (RIs)</i>	<i>Savings Plans (Compute)</i>
<i>Commitment</i>	"Box" Commitment	"Dollar" Commitment
<i>What you commit to</i>	<i>Specific Instance Type/Region</i>	<i>Hourly Spend (e.g., \$10/hour)</i>
<i>Duration</i>	1 or 3 years	1 or 3 years
<i>Max Discount</i>	Up to 72%	Up to 72% (Exact same rates)
<i>Applies to EC2?</i>	✓ Yes	✓ Yes
<i>Applies to Other Services?</i>	✗ No	✓ Yes (Huge advantage)
<i>What happens if you change OS Or m5a.large to m6g.large?</i>	Discount might stop	Automatically adapts
<i>Management Effort</i>	<i>High</i> (Requires manual swapping/modifications)	<i>Zero</i> (Completely automatic)

Spot Instances (The "Spare Capacity" Model)

- **How it works:** AWS sells its excess, unused data center capacity at a massive discount (up to **90% off**).
- **The Catch:** If AWS needs that capacity back for an On-Demand user, you get a **2-minute warning**, and your instance is terminated.
- **Best for:** Stateless applications, batch processing, background data analysis, or CI/CD testing pipelines.
- Not Suitable for critical and Databases

Dedicated Hosts(The Private House)

- **How it works:** You rent an entire physical server dedicated exclusively to your use.
- **Best for:** Strict corporate compliance or when you have "Bring Your Own License" (BYOL) software (like certain Windows Server or Oracle licenses) that ties to physical cores.
- **Use Case : Strict Regulatory Compliance**
 - Certain industries—like defense, banking, healthcare, or government agencies—have strict data privacy laws
 - **The Rule:** Some regulations state that your data *cannot* sit on the same physical hard drive or CPU chip as another company's data.

Dedicated Instances (The Private Floor)

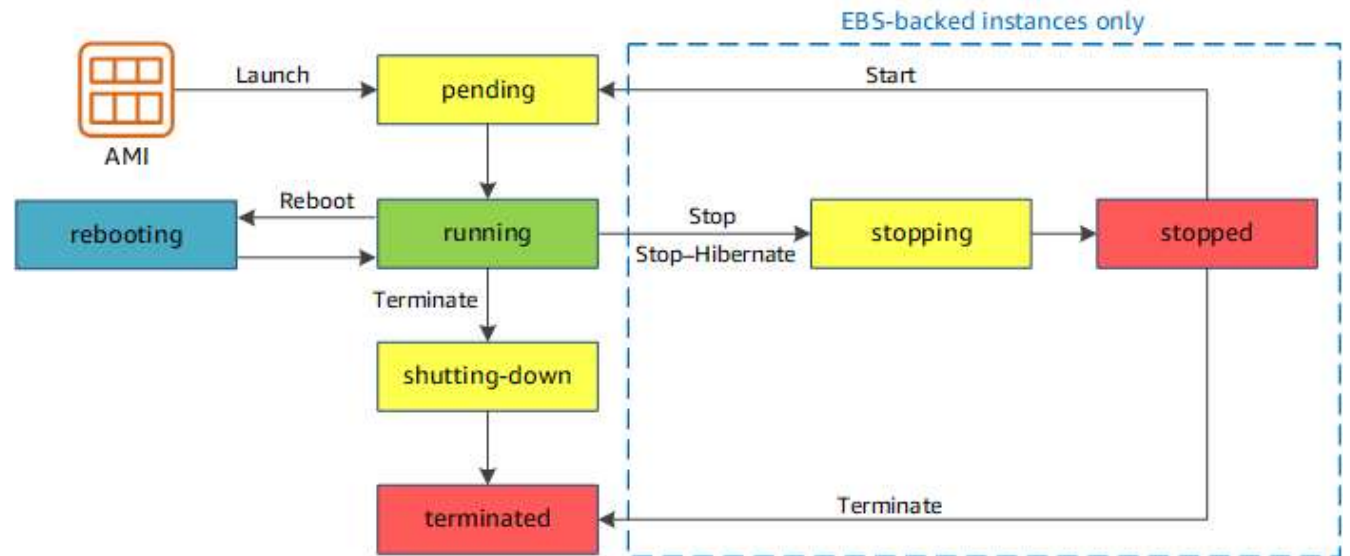
- **Who it work:** you tell AWS: "I want a room, but I don't want any strangers in my building."
- AWS takes a physical server rack and dedicates it to your AWS account.
- No other AWS customers can put their virtual machines on that physical box.
- **The Catch:** AWS still fully controls that server. If you turn your instance off and turn it back on, AWS's automated system might move your virtual machine to Room 205 on a completely different private floor.
- You don't know which physical machine you are on; you just have AWS's promise that you aren't sharing the physical hardware with strangers.

Capacity Reservations (Guarantee)

- **Availability Guarantee:** A feature that reserves physical compute capacity (CPU, RAM, and hardware) in a specific **Availability Zone (AZ)** for your EC2 instances.
- **Risk Mitigation:** Prevents "On-Demand capacity insufficiency errors" during high-traffic events, ensuring AWS always has a physical server ready for you.
- You are billed the full On-Demand price for the capacity **whether you launch an instance into it or leave it empty.**
- You can create a reservation for an hour, a day, or a month, and delete it whenever you want. **No long-term commitment** (1 or 3 years)

EC2 instance Lifecycle

EC2 instance Lifecycle



Pending

- **What it means:** The temporary state right after you launch an instance or start a stopped instance. AWS is currently locating the physical host hardware and provisioning the virtual machine.
- **Billing: Free.** You are not charged while the instance is pending.

Running

- **What it means:** Your instance is fully booted, functional, and ready to handle traffic.
- **Billing: Charged.** You pay per-second or per-hour for the compute capacity and any public IPv4 addresses assigned to it.

Stopping

- **What it means:** A temporary transition state when you choose to shut down your instance. The operating system is performing a graceful shutdown.
- **Billing: Free for compute**, but you are still billed for the attached EBS storage volumes.

Stopped

- **What it means:** The instance is shut down and completely powered off. Think of this like turning off your physical laptop. AWS releases the underlying host hardware.
- **Billing: Free for compute (\$0/hour).** However, you **must still pay for the attached EBS storage volumes** because your data is still occupying space in the data center.

Shutting-Down

- **What it means:** A brief transition state when you choose to "Terminate" (delete) your instance. AWS is cleaning up resources and preparing to wipe the environment.
- **Billing: Free.**

Terminated

- **What it means:** The instance is permanently deleted. It will remain visible in your AWS console dashboard for a few hours before disappearing completely. **You cannot recover or restart a terminated instance.**
- **Billing: Free.** All compute and standard attached EBS billing stops immediately.

HIBERNATED

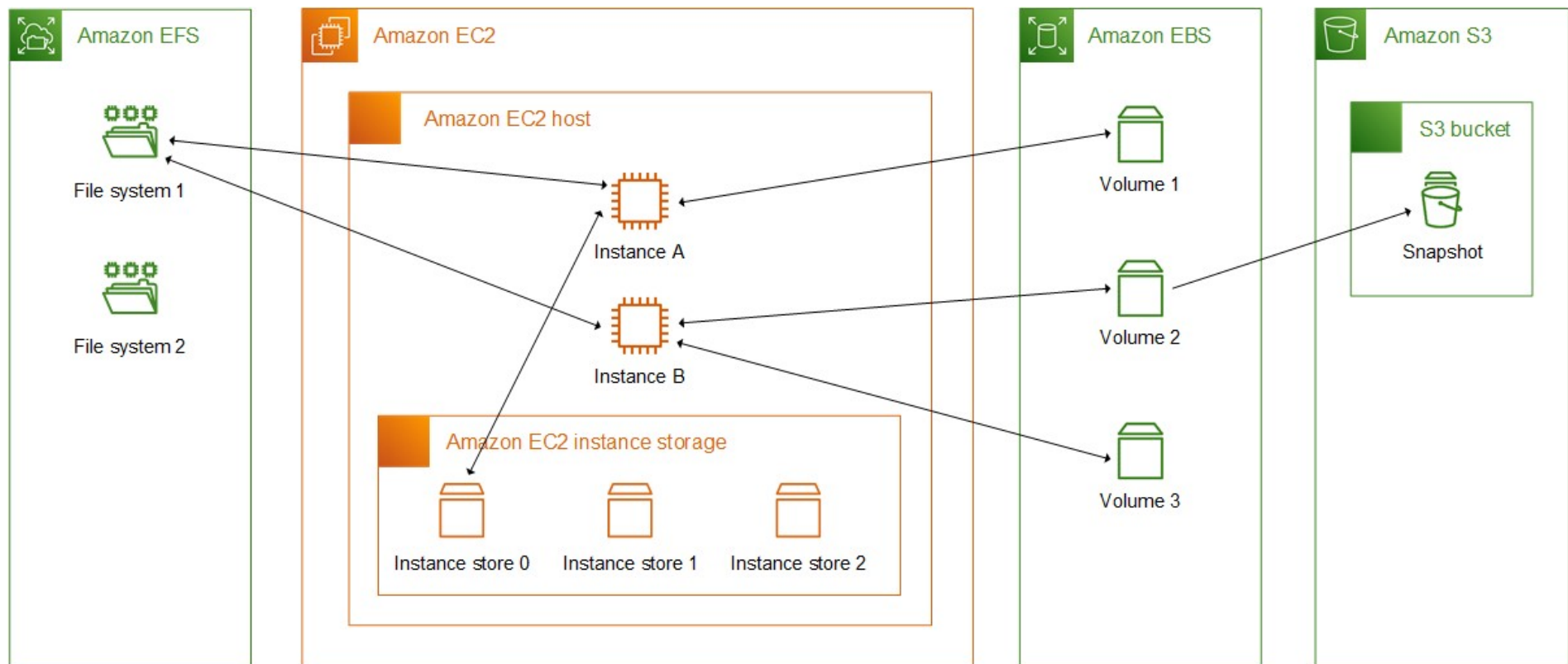
- **What happens :** AWS takes everything currently sitting in your instance's RAM (open terminal sessions, running application states, background tasks) and **writes it directly onto your EBS root volume** as a giant hidden file.
- **The Freeze:** Once the RAM is safely saved onto the EBS drive, the instance powers down.
- **The Unfreeze (Restart):** When you start the instance back up, AWS reads that file from the EBS volume, reloads it into the physical RAM, and your server wakes up *exactly* where you left it in a few seconds.
- **Billing:** Just like stopping, compute fees drop to \$0/hour. However, because your EBS volume now has to hold both your regular data *and* the giant RAM file, **your EBS storage costs will be slightly higher** during hibernation to accommodate that extra RAM data footprint.

AWS – EC2 Storage

What is Storage?

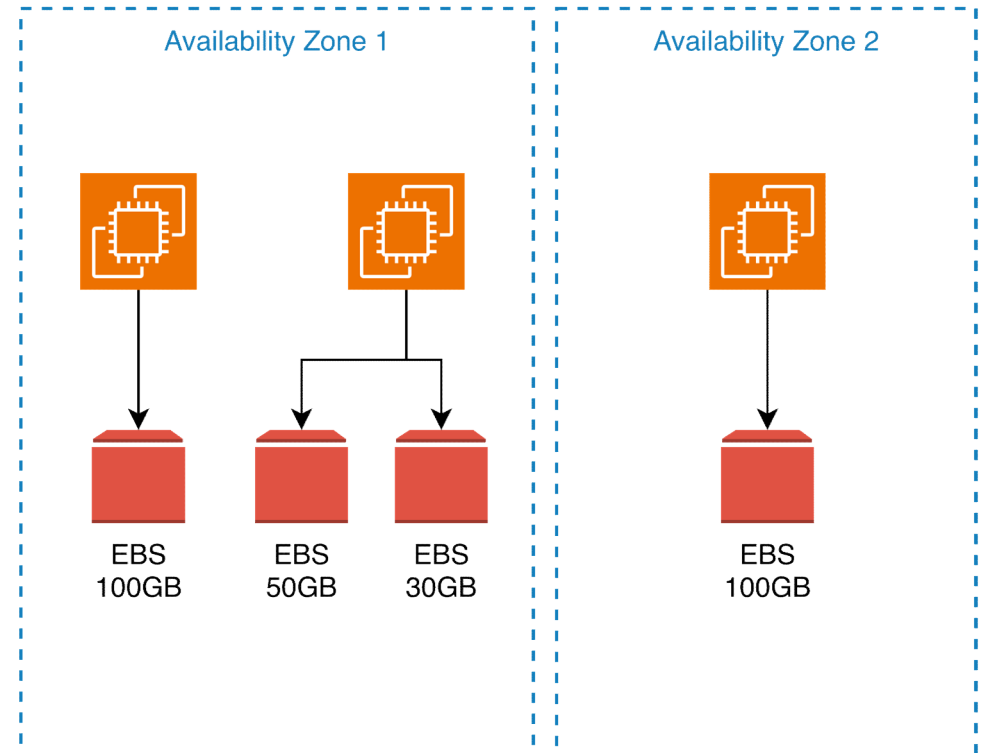
- **AWS Storage** is a cloud-based services that replace physical hard drives, on-premises file servers, and tape backup systems. Instead of buying physical disks, you provision exactly what you need over the internet.
- Three primary types:
 - Block Storage (The Local Hard Drive)
 - EBS
 - Instance Store
 - File Storage (The Shared Network Folder)
 - EFS
 - FSx
 - Object Storage (The Infinite Bucket)
 - S3

AWS Storage



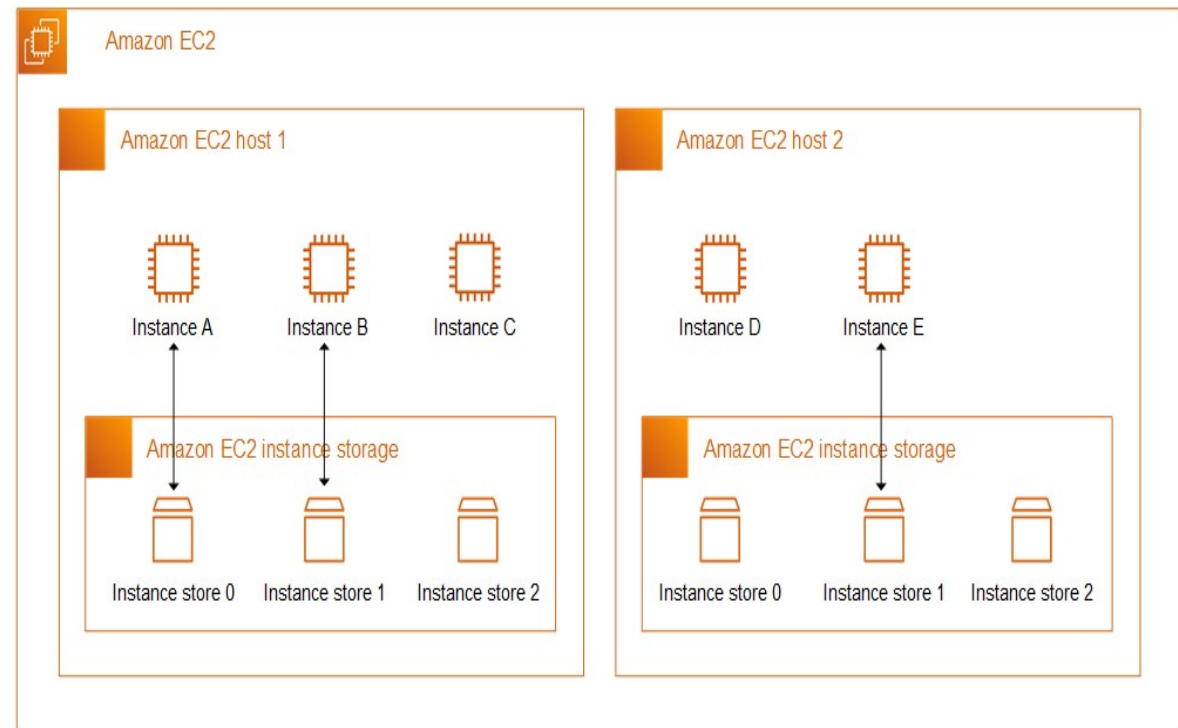
Amazon EBS (Elastic Block Store)

- Think of this as a virtual external hard drive that you plug directly into the back of a single virtual machine (EC2 instance).
- It bypasses standard IP networking and connects via a dedicated storage channel, giving it ultra-low latency.
- It is **locked to a single Availability Zone (AZ)**.
- EC2 OS (Linux/Windows) is usually stored on this system and used for booting up.
- Used for running high-performance databases (like Oracle or MySQL) that need lightning-fast read/write speeds.
- Typically attaches to only one EC2 instance at a time.
- Data survives even if the EC2 instance is **Stopped** or **Shut down**. You pay for the storage space as long as the volume exists.
- It can be detached from an EC2 instance and attached to another one quickly.



EC2 Instance Store (The Ephemeral Alternative)

- Built directly into the physical host hardware motherboard running your EC2 instance.
- Bypasses the network layer entirely, providing the **lowest possible latency and highest I/O speeds**.
- Data is **permanently deleted** if the instance is **Stopped** or **Terminated**. Data only survives a regular operating system reboot. Thus Ephemeral (Temporary).
- Used for High-speed caching, temporary swap space, or "scratch pads" where data loss does not impact the application.
- By default, for most standard instances (like t2.micro, t3.medium, or m5.large), an Instance Store is **not** attached.
- Certain specific instance types come with physical SSD drives physically welded into the instance store of host server.
- Can spot these by looking for an **"d"** in the instance name example: **m5d.large**, **c5d.xlarge**, or **i3.enormous**.



Difference EBS vs Instance Store

Feature	EBS Volume (Attached by Default)	Instance Store (Attached on 'd' series)
Physical Location	A separate network storage device.	Physically inside the host server hardware.
Data Persistence	Data survives if you Stop and Start the instance.	Data is permanently deleted if you Stop or Terminate the instance.
Performance	Fast (Limited by network bandwidth).	Ultra-Fast (Incredibly high I/O, lowest latency).
Use Case	General OS boot, long-term files, regular databases.	Caching layers, temporary scratch pads, swap space.

EBS Volume Types

- **gp3 / gp2 (General Purpose SSD):** The default choice for virtual servers. Balances price and performance. gp3 allows you to scale speed (IOPS) and size independently.
- **io2 / io1 (Provisioned IOPS SSD):** High-performance, low-latency storage. Mandatory exam choice for **critical relational databases** (SQL Server, Oracle) with high transaction volumes.
- **st1 (Throughput Optimized HDD):** Low-cost, fast sequential streaming storage. Best for **big data logs, MapReduce, and data warehousing**. *Cannot be used as a boot volume.*
- **sc1 (Cold HDD):** The absolute cheapest EBS tier. Built for **infrequently accessed file archives**. *Cannot be used as a boot volume.*

Comparison

Type	Storage	Performance	Cost	Best For
gp3	SSD	High	Medium	General purpose
gp2	SSD	Medium	Medium	Older workloads
io1/io2	SSD	Very High	High	Databases
st1	HDD	High Throughput	Low	Big data
sc1	HDD	Low	Very Low	Archive/backup

<https://docs.aws.amazon.com/ebs/latest/userguide/ebs-volume-types.html>

Important Notes

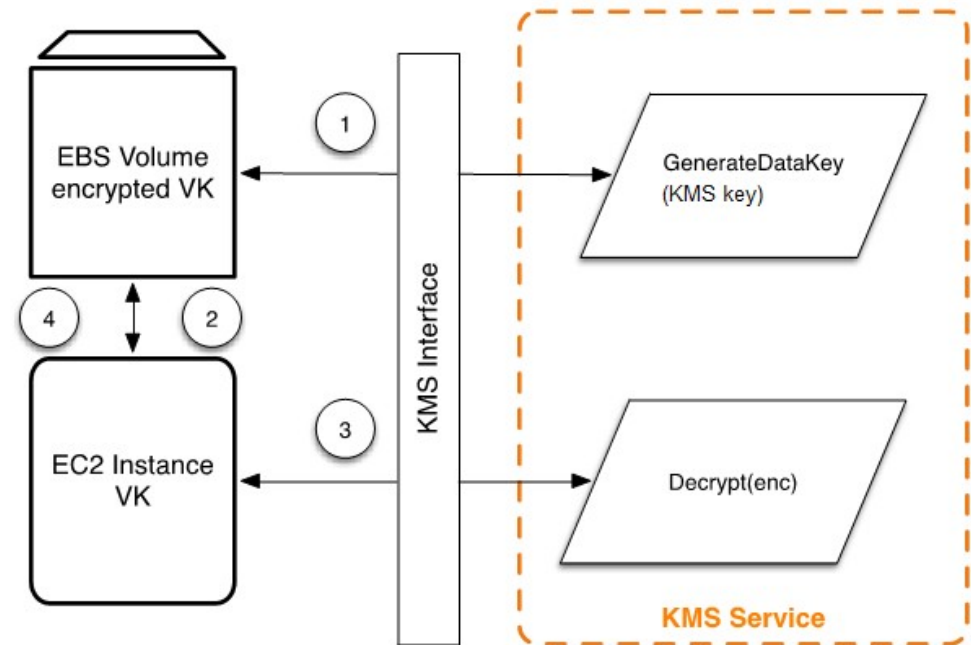
- **gp3** is the recommended default EBS type.
- **io1/io2** are used when high IOPS is required.
- **st1** and **sc1** cannot be used as boot volumes.
- EBS volumes are **AZ-specific**.
- EBS snapshots are stored in Amazon Web Services Amazon S3 internally.
- Root EBS volumes are deleted automatically by default when the EC2 instance is terminated.

EBS Volume Encryption

- Amazon Elastic Block Store (EBS) Encryption protects data stored in EBS volumes by encrypting it at rest and in transit between the EBS volume and the EC2 instance.
- Encryption is handled automatically by Amazon Web Services using AWS Key Management Service (AWS KMS).
- When EBS encryption is enabled, the following components are encrypted automatically:
 - EBS volume data
 - Snapshots created from encrypted volumes
 - Data moving between EC2 and EBS
 - Volumes created from encrypted snapshots
- When you create an encrypted volume:
 - AWS uses a KMS key to encrypt data
 - Data is automatically decrypted when accessed by EC2

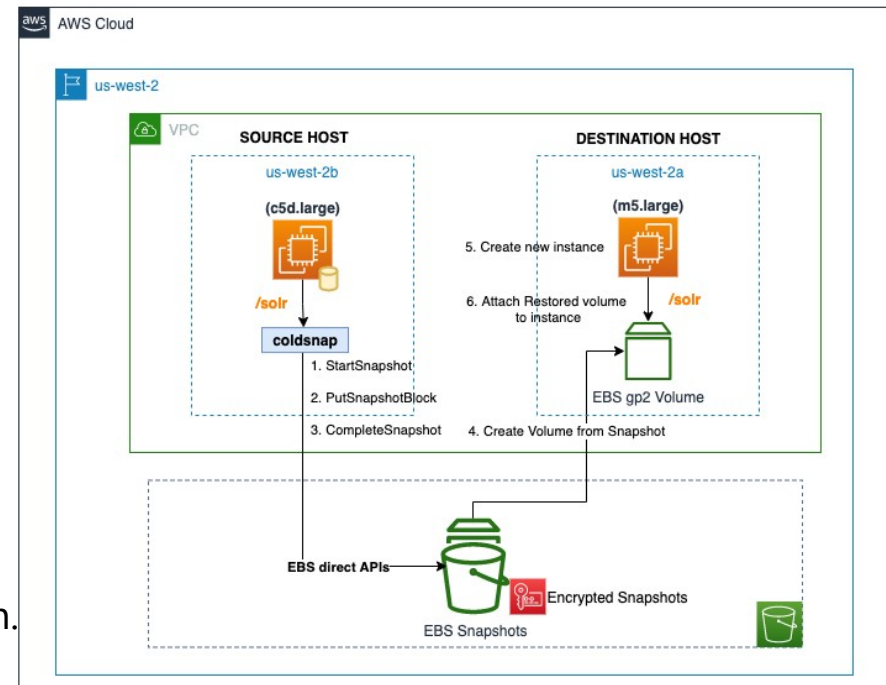
KMS Role in EBS Encryption

- AWS Key Management Service (KMS) is a managed service used to create and control encryption keys.
- AWS Managed Key → Default key (AWS handles everything)
- Customer Managed Key (CMK) → User controls permissions and rotation
- How EBS uses it:
 - KMS → creates **Master Key (CMK)**
 - EBS → requests **Data Key from KMS**
 - Data Key → encrypts actual EBS data
 - Data Key itself is protected by CMK



EBS Snapshot

- A **backup copy of an EBS volume**.
- Stored in Amazon S3 internally (managed by Amazon Web Services).
- Point-in-time capture of a volume.
- Used for:
 - Backup
 - Disaster recovery
 - Creating new volumes
- EBS Snapshots are Incremental, After the first full snapshot, only the changed blocks are saved in the next snapshots.
- The very first snapshot copies **all data blocks** from the EBS volume. Second snapshot, Only changed/new blocks since the previous snapshot are stored.



EBS Common scenario

Scenario 1: The Accidental Termination

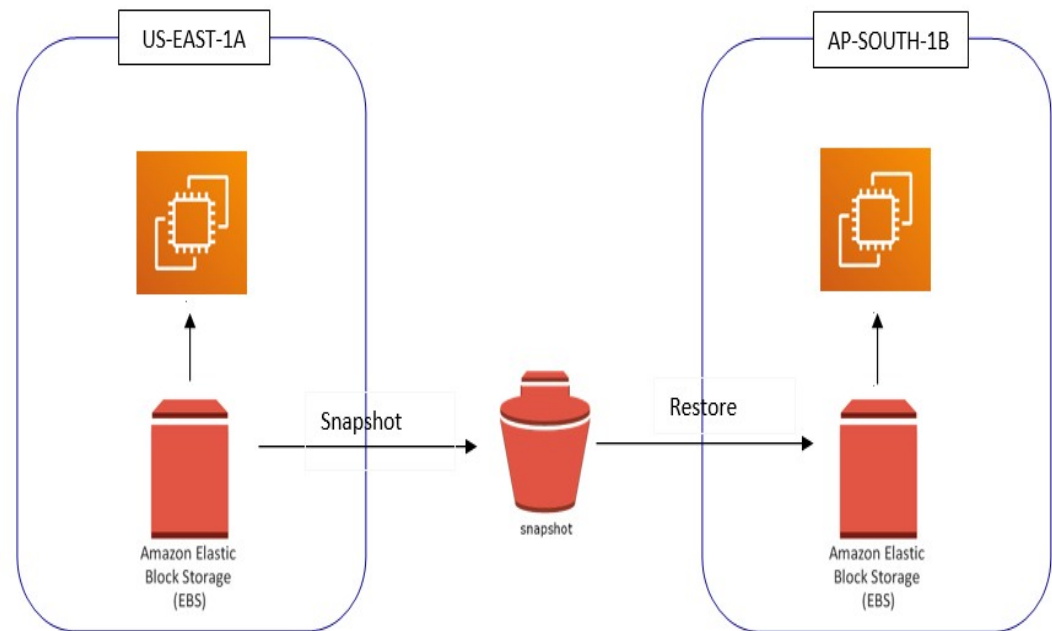
- An administrator accidentally terminates an EC2 instance running a critical application. The instance used default storage settings. The root EBS volume is gone. How do you restore the application to a working state with minimal data loss?
- **The Core Problem:** Root volumes default to Delete on Termination = True. The drive was wiped the instant the instance died.
- **Solution:** Controls the EBS behavior when an EC2 instance terminates
 - By default, the root EBS volume is deleted (attribute enabled)
 - By default, any other attached EBS volume is not deleted (attribute disabled)
 - EBS volume is floating in your account as an independent unattached drive.

Scenario 2: The "Orphaned Volume"

- A company spin up EC2 instances during high-traffic hours. Each instance attaches a secondary EBS volume to store application logs. The company notices that while instances are successfully terminated when traffic drops, their monthly AWS EBS bill keeps rising drastically. What is causing this?
- **The Core Problem:** Secondary (non-root) EBS volumes default to Delete on Termination = False. When they terminates the instances, the servers disappear, but the log drives are left behind as unattached, "orphaned" volumes that continue to incur full storage fees.

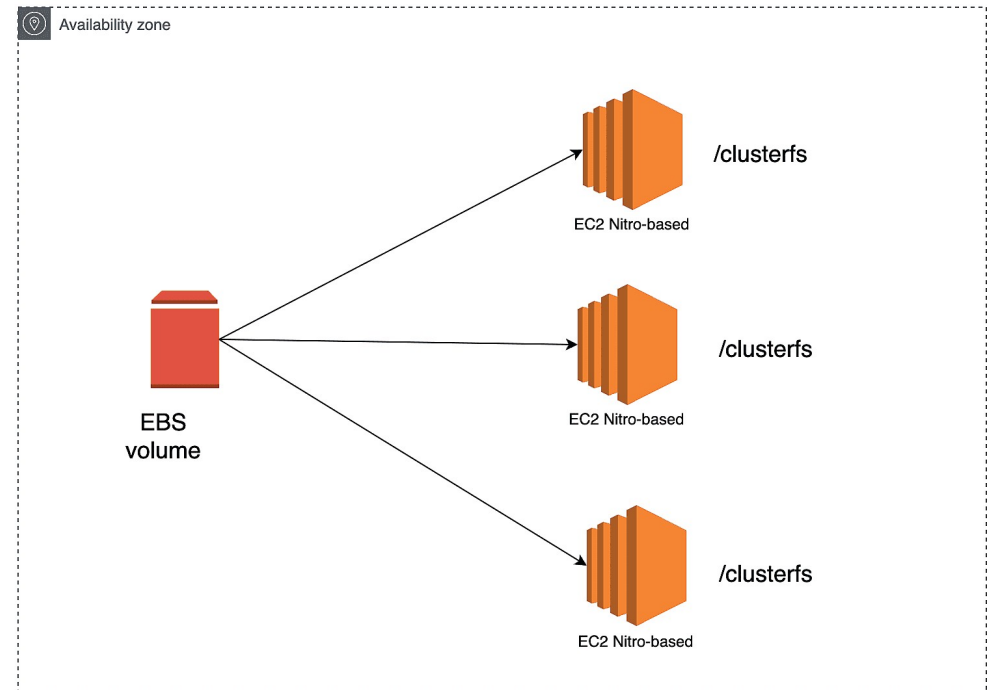
Scenario 3: Moving a Database to a Different AZ

- You have a high-performance database running on an EC2 instance with an EBS volume in us-east-1a. Due to a corporate compliance change, you need to migrate this database to us-east-1b with minimal downtime and zero data corruption. How do you achieve this?
- **The Core Problem:** EBS volumes are **AZ-Locked**. You cannot simply unplug a drive from us-east-1a and plug it into a server in us-east-1b.
- **The Solution using Snapshots:**
 - Stop the database application on the EC2 instance to ensure no new data is being written (ensuring "data consistency").
 - Create an **EBS Snapshot** of the database volume. (Snapshots are stored in S3, making them region-wide, not AZ-locked).
 - Go to the Snapshots menu, select your snapshot, and click **Create Volume**.
 - In the settings, change the target Availability Zone to **us-east-1b**.
 - Launch your new database EC2 instance in us-east-1b and attach this new volume to it.



EBS Multi-Attach

- **EBS Multi-Attach** is a feature that allows **one EBS volume** to be attached to **multiple EC2 instances simultaneously**.
- Supported only on:
 - **io1**
 - **io2** EBS volumes
- Can attach Up to **16 Nitro-based EC2 instances**.
- All instances must be in the **same Availability Zone (AZ)**.
- You **cannot** format a Multi-Attach volume with traditional standard filesystems like **XFS, EXT4, or NTFS** and allow multiple instances to write to it at the same time. Doing this will result in immediate, catastrophic data corruption.
- Use a **Cluster-Aware Filesystem**(like GFS2 or OCFS2 on Linux). These specialized filesystems include a feature called a **Distributed Lock Manager (DLM)**. Before Server A is allowed to write to Sector 500, it sends a network message to Server B saying: *"Hey, I am locking Sector 500 right now. Do not touch it."* Once Server A is finished, it releases the lock, and Server B is allowed to read or write to it safely.



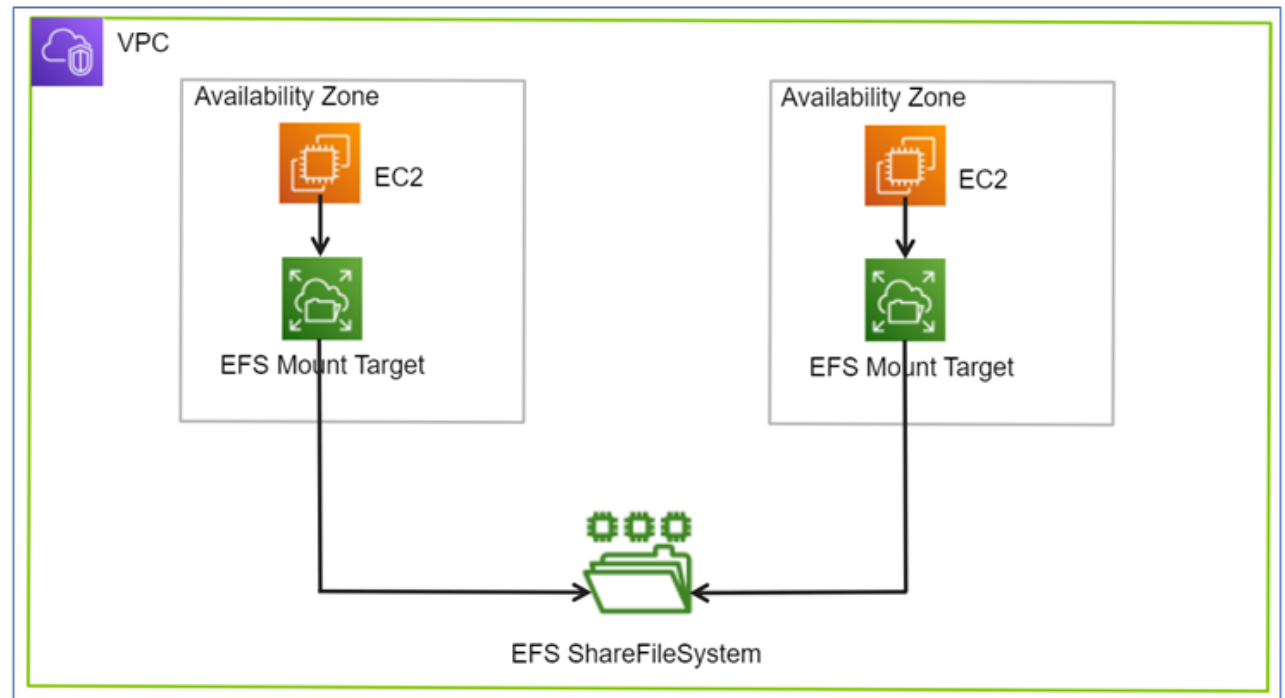
Limitations of EBS

- EBS is Limited to One Availability Zone.
- EBS Usually Attaches to One EC2 Instance.
- EBS Cannot Be Accessed Over Internet/API Like S3.
- EBS is Block Storage, so it's a raw block device You must format filesystem, manage sharing & locks many technical issues thus not shared between the device easily.
- EBS Scaling is Manual and not auto scales automatically.

Amazon EFS

Amazon EFS (Elastic File System)

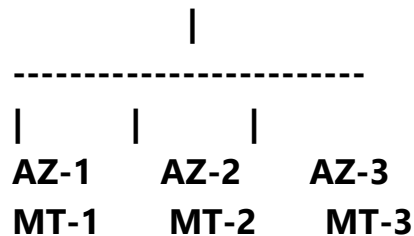
- **EFS (Elastic File System)** is a Fully managed, **scalable network file storage service** for EC2 instances.
- It provides a **shared file system** that multiple servers can use at the same time.
- It uses NFS (Network File System) (NFSv4)4.1 protocol.



How EFS works

- Amazon **EFS (Elastic File System)** works as a **managed network file system** that multiple EC2 instances can access at the same time using **NFS (Network File System)**.
- **Once we create** EFS file system AWS provisions a managed distributed file system which is auto-scales.
- After EFS creation how to reach these file system from EC2 thus we creates Mount Targets (per AZ).
- Mount target has a **private IP inside a subnet in AZ which is backend mapped to EFS**.

EFS File System



- EC2 sends request via **NFS protocol**, Traffic goes through:

EC2>>>Security Group>>>Mount Target>>>EFS backend storage

EFS Key Features

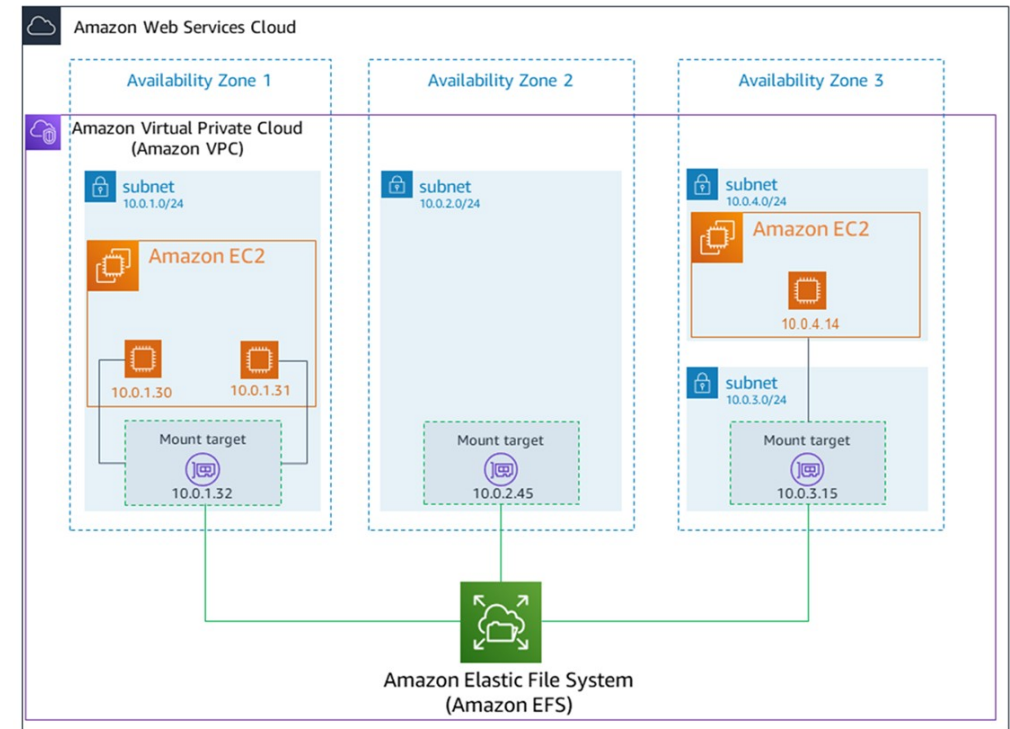
- Multiple EC2 instances can read/write at the same time.
- Automatically spreads data across multiple Availability Zones thus High availability by default.
- Fully Managed, AWS handles scaling, patching and replications.
- Auto Scaling, Grows automatically as you add files and Shrinks when you remove data.
- Works mainly with Linux EC2 instances.
- Real-Life Analogy EFS = Google Drive.

EFS Storage Classes

- EFS categorizes its storage classes based on two main factors: **Access Frequency** (how often you need the data) and **Redundancy** (how the data is replicated).
- 1. Standard Storage Classes (Multi-AZ): These classes replicate your data across multiple Availability Zones (AZs) within an AWS Region. They offer the highest level of availability and durability.
 1. **Amazon EFS Standard:** Designed for active, frequently accessed data. It delivers the lowest latencies for read and write operations.
 2. **Amazon EFS Standard-Infrequent Access (Standard-IA):** Cost-optimized for data that is not accessed every day. You pay a lower storage price but incur a per-GB fee when you read or write to it.
 3. **Amazon EFS Standard-Archive:** Optimized for long-term, rarely accessed data (accessed a few times a year or less). It offers the lowest storage cost for Multi-AZ replication, with higher retrieval fees.

Multi-AZ EFS

- A fully managed file system that stores data **across multiple Availability Zones (AZs)**.
- Data replicated across **multiple Azs**.
- Very high **availability and** durability
- Survives AZ failure.
- Requires mount targets in multiple Azs.
- Use cases Production workloads or Critical applications or High availability systems.



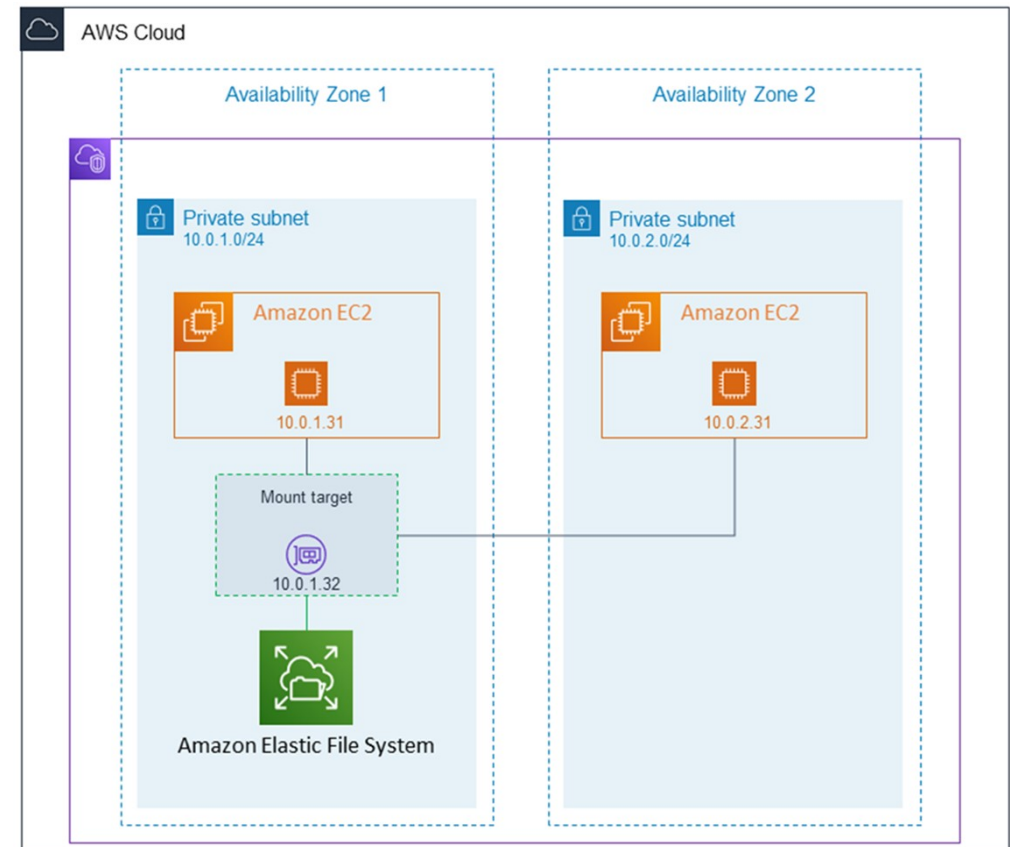
EFS Storage Classes

2. One Zone Storage Classes (Single-AZ): These classes store data redundantly within a *single* Availability Zone. They are ideal for development workloads, backups, or data that can be easily recreated, offering up to **47% savings** compared to Standard classes.

1. **Amazon EFS One Zone:** For frequently accessed data stored in a single AZ.
2. **Amazon EFS One Zone-Infrequent Access (One Zone-IA):** For infrequently accessed data stored in a single AZ.

One Zone EFS

- A file system that stores data in **only one Availability Zone**.
- Lower cost than Regional EFS.
- No cross-AZ redundancy
- Still fully managed by AWS
- Requires mount target only in that AZ
- Use Case Non-critical workloads or Temporary storage.



Amazon EFS – Storage Classes

- EFS Standard (Frequent Access) is Default storage class for EFS.
 - Best for frequently used files
 - High performance and Multi-AZ storage.
- EFS Infrequent Access (EFS-IA) is Lower-cost storage for files that are not accessed often.
 - Automatically or manually moved (via lifecycle policy).
 - Cheaper than Standard.
 - Slightly higher access latency.
- EFS Archive is Lowest-cost storage class for long-term storage.
 - Cheapest storage option
 - Highest retrieval latency
 - Good for compliance and archival data
- Amazon EFS storage classes (Standard, Infrequent Access, and Archive) help optimize cost by automatically or manually moving files based on how frequently they are accessed.
- EFS can automatically move files between classes:
 - Standard → (after 30/60/90 days) → Infrequent Access → Archive

EFS Security

- Data stored in EFS is encrypted Uses **AWS KMS (Key Management Service) for encryption.**
- Data encrypted between EC2 and EFS during transit uses TLS for same.
- You can Control who can create, delete or modify EFS why IAM policies.
- Security Groups controls Network access to mount targets need to explicitly allow **port 2049.**
- EFS is accessible only inside your VPC, No public internet access
- POSIX Permissions Standard Linux file permissions to controls file-level access.

IMP EFS Points

- As Uses network based **NFS protocol it uses (port 2049) for its communication thus we need to allow it in SG of EC2.**
- Multiple EC2 instances can access same files simultaneously.
- Storage grows and shrinks automatically.
- Works mainly with Linux EC2

EFS Lifecycle Management

- EFS uses **Lifecycle Management** to monitor access patterns and automatically transition files. You don't have to manually move files between these storage classes.
 - **Transition to IA/Archive:** You can set a policy (e.g., if a file hasn't been accessed for 30 days, move it to Standard-IA; if it hasn't been accessed for 90 days, move it to Archive).
 - **Transition back to Standard:** You can enable **Lifecycle transition back to Primary storage**, which automatically moves a file back to EFS Standard/One Zone the moment someone accesses it, ensuring subsequent reads are fast and don't incur repeat IA/Archive retrieval fees.
- When you create a brand new EFS file system. Under **Availability and Durability**, choose either **Regional** (Standard) or **One Zone**. You cannot convert a Standard file system to a One Zone file system later.

EFS vs EBS

Feature	EFS	EBS
Type	File Storage	Block Storage
Access	Multiple EC2	Single EC2
AZ	Multi-AZ	Single AZ
Scaling	Auto	Manual
Protocol	NFS	Block device

Accessing Instances

Accessing Instances

- A **Bastion Host** (also commonly called a "Jump Box") is a **design pattern** where you use a standard **EC2 instance** to act as a secure gateway for accessing your private servers.
- As its not a service, you are responsible for configuring the security groups, the SSH keys, and the OS hardening.
- Must reside in a **Public Subnet** with an **Elastic IP** (so the IP doesn't change when you reboot it)
- The Bastion's Security Group should **only** allow Port 22 (SSH) or 3389 (RDP) from **your specific office IP**. Never open it to 0.0.0.0/0
- You should disable password-based login and use only SSH Key Pairs.
- Modern Alternatives: AWS Systems Manager (SSM) Session Manager & EC2 Instance Connect Endpoint

